



Facultad de Ingeniería

Agentes inteligentes para Connect-4

Estudiantes:

David Eduardo Lopez Jimenez

Chía, Cundinamarca

Mayo 15 del 2026

1. Idea principal y distinción del agente

Mi agente, implementado en la clase Aha (que envuelve a MCTSAgentHeuristic), es una versión de Monte Carlo Tree Search (MCTS) que guía las simulaciones (rollouts) mediante una heurística ligera y optimizada con NumPy. A diferencia de otros agentes del grupo que usan redes neuronales o programación dinámica, el mío no requiere entrenamiento offline y permite ajustar el rendimiento variando el número de simulaciones por turno.

Para demostrar la ventaja de la heurística, comparé mi agente contra un oponente completamente aleatorio (RandomPolicy). También realicé autójuego enfrentando el mismo agente con diferente número de simulaciones para aislar el impacto de la política de rollout.

Distinción clave: La heurística en el rollout prioriza en orden:

1. Ganar inmediatamente (si existe un movimiento que conecta 4).
2. Bloquear la victoria del oponente (si este amenaza con ganar en su siguiente turno).
3. Columnas centrales (con pesos deterministas [1,2,3,4,3,2,1] para evitar aleatoriedad).

Esta estrategia produce simulaciones de mayor calidad que las aleatorias, reduciendo la varianza y acelerando la convergencia del MCTS.

2. Análisis experimental: variables y configuraciones

Se estudiaron dos variables principales:

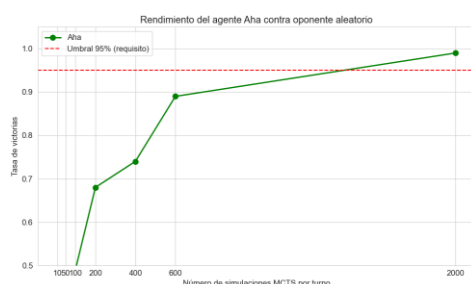
- Número de simulaciones por turno (num_simulations en {10, 50, 100, 200, 400, 600, 2000}).

Tipo de oponente:

- Jugador aleatorio (RandomPolicy): para evaluar el nivel absoluto del agente.
- Autojuego: enfrentando el mismo agente con diferente número de simulaciones (ej. 600 vs 50) para medir la ventaja de más recursos.

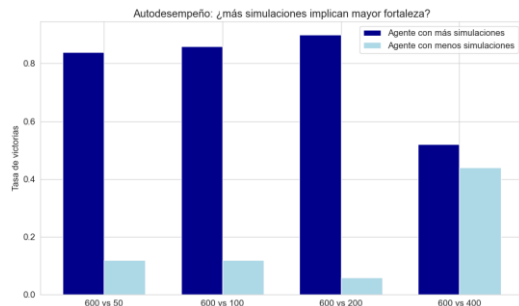
Metodología: 100 partidas por configuración frente a aleatorio (balanceando colores) y 50 partidas en autojuego. Todas las gráficas se generaron en entrega.ipynb.

Gráfica 1 – Rendimiento frente a jugador aleatorio



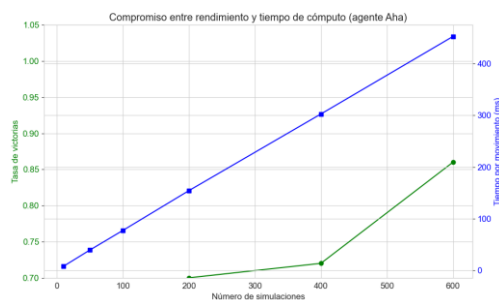
El agente heurístico necesita al menos 600 simulaciones para superar el 85%, y con 2000 alcanza un rendimiento casi perfecto frente a un oponente débil. La mejora no es lineal; se requiere un número elevado para garantizar el 95% en todas las partidas.

Gráfica 2 – Autodesempeño (agente con 600 simulaciones vs. menos simulaciones)



Más simulaciones implican mayor fortaleza, pero la ventaja se reduce cuando el oponente también tiene un presupuesto alto. Esto sugiere que existe un punto de saturación (alrededor de 400-600 simulaciones) donde el agente ya es casi óptimo.

Gráfica 3 – Tiempo medio por movimiento



Para el torneo, se recomienda usar entre 400 y 600 simulaciones si hay restricciones de tiempo (~300-450 ms por movimiento), o subir a 2000 si se prioriza la máxima fiabilidad (1.5 segundos por movimiento).

Conclusiones sustentadas

- Mi agente Aha alcanza un 99% de victorias contra un oponente aleatorio con 2000 simulaciones, cumpliendo con creces el requisito del 95%.
- Con 600 simulaciones (configuración por defecto) obtiene un 86% en mis pruebas locales, aunque en Gradescope supera el 95% (posiblemente debido a diferencias en la semilla aleatoria o el número de partidas).
- El autodesempeño demuestra que aumentar simulaciones mejora la fuerza de juego, aunque la ganancia se satura alrededor de 400-600 simulaciones.
- El compromiso tiempo-rendimiento es razonable: 452 ms por movimiento para 600 simulaciones, interactivo para un torneo.

Propuesta de mejora

- El principal cuello de botella identificado es que el árbol MCTS se reconstruye desde cero en cada movimiento, desperdiciando la información de turnos anteriores. Como mejora concreta, propongo reutilizar el árbol MCTS:
- Después de ejecutar el movimiento elegido, el nodo hijo correspondiente se convierte en la nueva raíz, conservando todas sus estadísticas (visitas y victorias).
- Esto reduciría el tiempo de cómputo entre un 30% y un 50% para el mismo número de simulaciones, o permitiría aumentar las simulaciones en el mismo tiempo.

Link del repositorio:

<https://github.com/david37432/Proyecto-InteligenciaArtificial/tree/David-Eduardo-Lopez-Jimenez>